

A Web Framework For Test Automation: User Scenarios Through User Interaction Diagrams

Douglas Hiura Longo

Informatics and Statics Department,
Postgraduate Program in Computer Science (PPGCC)
Federal University of Santa Catarina
Florianopolis, Brazil
+55 49 9903-2569
doughlashiura@posgrad.ufsc.br

Patricia Vilain

Informatics and Statics Department,
Postgraduate Program in Computer Science (PPGCC)
Federal University of Santa Catarina
Florianopolis, Brazil
+55 48 9980-2621
patricia.vilain@ufsc.br

Lucas Pereira da Silva

Informatics and Statics Department,
Postgraduate Program in Computer Science (PPGCC)
Federal University of Santa Catarina
Florianopolis, Brazil
+55 47 9711-1754
lucas.pereira@posgrad.ufsc.br

Ronaldo dos Santos Mello

Postgraduate Program in Computer Science (PPGCC)
P. P. in Methods and Management in Evaluation
(PPGMGA)
Federal University of Santa Catarina
Florianopolis, SC, Brazil
r.mello@ufsc.br

ABSTRACT

This paper presents a web framework for test automation. This framework automates tests specified using the technique User Scenarios through User Interaction Diagrams (US-UID). US-UIDs represent customer requirements and are created a priori to the development of the application code. The goal of this proposal is the execution of US-UIDs as acceptance tests to check the application code in the context of Acceptance Test-Driven Development (ATDD). The implementation of the proposed framework uses FitNesse as its base and adds an editor for creating and executing US-UIDs. The proposal is exemplified by US-UIDs of the 8-puzzle game and evaluates the code of a Web-based application. The results show that US-UID has an executable format as automated tests. In addition, the proposed web framework has the capability of indicating the success, error or failure of interaction between a US-UID and the application code.

Categories and Subject Descriptors

D.2.5 [Software Engineering]: Testing and Debugging - Test tools.

D.2.1 [Software Engineering]: Requirements/Specifications – Tool.

General Terms

Verification.

Keywords

US-UID; User Scenario; User Interaction Diagram; UID; Acceptance Test; ATDD; Automated Test; FitNesse.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

iiWAS '16, November 28-30, 2016, Singapore, Singapore.

© 2016 ACM. ISBN 978-1-4503-4807-2/16/11...\$15.00

DOI: <http://dx.doi.org/10.1145/3011141.3011158>

1. INTRODUCTION

Analogous to the Test-Driven Development (TDD) approach [1], Acceptance Test-Driven Development (ATDD) consists of the use of Automated Acceptance Tests (AAT) with the additional constraint that these tests are written in advance of implementing the application code [2, 3]. The basic idea of AAT is to document requirements and expected outcomes in a format that can be automatically and repeatedly tested [4]. AAT represents customer expectations [5] and is adopted in agile software development holding great promise of improving communication and collaboration among those involved [4, 6, 7].

Alvestad [8] investigated whether a non-technical customer could express requirements based on domain specific languages, but was not successful in confirming his assumptions. However, according to Longo and Vilain [9], US-UIDs are a representation format for software requirements. The US-UIDs enable the creation and representation of software requirements for non-technical customers a priori to the development of the application code [9].

In this paper we propose a web framework that uses US-UIDs to create automated tests whose traceability indicates the success, error or failure of the application code. Moreover, the proposal evaluates the feasibility of Web-based application through US-UIDs execution as automated testing.

This paper is organized as follows: Section 2 presents the background. Section 3 presents details of the proposal and implementation of the web framework. Section 4 presents its applicability by an example of a web application. Section 5 presents the evaluation and the proposed results. Finally, the conclusions are presented in Section 6.

2. BACKGROUND

This section introduces the concepts of FitNesse, UIDs and US-UID to better understand the proposal presented in Section 3.

2.1 FitNesse

FitNesse is a framework for automated testing [4, 7]. It is used to automate business rules and acceptance tests that are expressed in

tables [4, 7, 10, 11]. The information included in tables can be divided into two categories (see Figure 1) [11]:

- Functional data - name of classes, methods and fields that are connected to the test class (test fixture);
- Test data - declarations of inputs and expected outputs.

CalculateAge			
birthday	calculusAge()		
21.10.2000	9		
23.10.1991	17		
23.11.1986	22		
22.10.1986	23		
...	...		

Functional data

Test data

Figure 1. Data stored in FIT specification, and its classification [11].

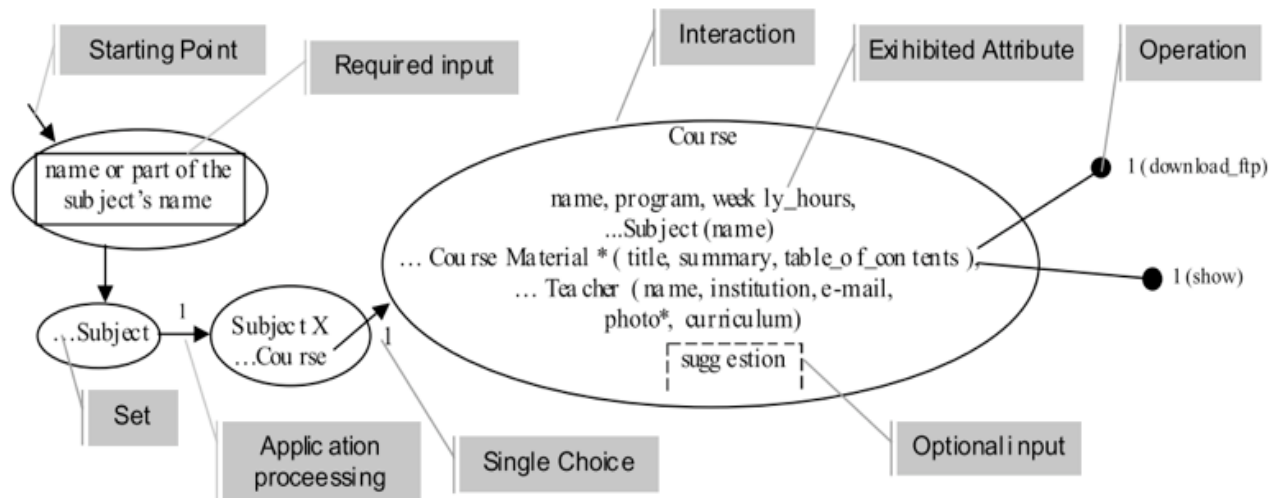


Figure 2. UID for the use case "Browsing courses given a subject" [13]

Figure 2 presents the elements of a UID, as follows:

- User input – a box that represents a data item provided by the user;
- System output – a text that represents a data item or system information (when represented with quotes) that is shown to the user;
- Transition – a link between states that shows the change of the interaction focus;
- Interaction State – an ellipse that represents the focus of the interaction between the user and the system and contains a set of user inputs, system outputs and/or transitions to other states.

2.3 User Scenarios through User Interaction Diagrams (US-UID)

US-UIDs also represent software application requirements. But they can be used by non-technical customers to create acceptance tests a priori to the development of the application code [9]. Figure 3 shows two examples of an 8-Puzzle game and the

According to Druk and Kropp [11], the functional data of FIT tables are structured with attributes and methods of a test class. This class is known as fixture. Fixture are responsible for the communication between test data and the System Under Test (SUT).

2.2 User Interaction Diagram (UID)

UIDs represent the interaction between the user and the system, without considering specific aspects of the user interface [12]. They can be used with traditional methods, such as scenarios and use cases, to collect requirements of an application [12, 13, 14]. Figure 2 shows the UID for the use case "Browsing courses given a subject" [13].

representation of a User Scenario through UID (US-UID). In both examples, the initial state is random; in the second state the user enters a value (->) to move the empty space; and the last state of the system displays the result [15].

In these examples the US-UIDs were created using the regressive method, where the interaction is shown from the final state to the initial state. As explained in [9], requirements specification as user scenarios can be created applying the following methods:

- Progressive (see Figure 3, left): US-UID is built following the flow of interaction as a procedure up to the final state. Figure 3 shows the progressive flow of the construction of a requirement specification, that is, every specification of the sum function is constructed, and its result is shown only at the end of the flow.
- Regressive (see Figure 3, right): similarly to the Assert-First technique [1], the regressive method starts the construction of the user scenario from the result.

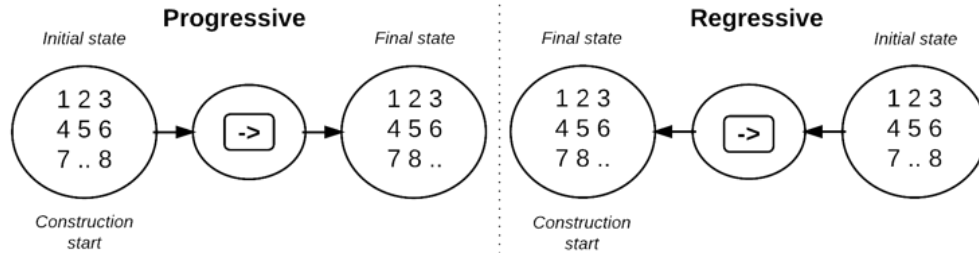


Figure 3. Example of US-UID of 8-Puzzle [15].

The US-UIDs are built with the information of the interactions between the user and the system and are structured with a language of symbols. The symbols of language are based on UID notation. Table 1 shows the subset of UID symbols that is used to create US-UID.

Table 1 - Symbols used by the US-UID [9]

Symbol	Use
	Ellipse - represents a state of interaction.
	Arrowed line - represents the transition between interaction states and flow direction.
	Rectangle – represents the user input, its value is represented by a set of characters contained within the ellipse.
Characters sequence	Value - represents the system output, where a set of characters is contained within the ellipse.

It is important to point out that although the symbols used in US-UID are a subset of UID symbols, they are used with different purposes in both techniques:

- In UID: used to structure the names of the model;
- In US-UID: used to structure the values and information of the scenario.

To exemplify this difference, Figure 4 shows a US-UID and its corresponding UID of an 8-puzzle game [9]

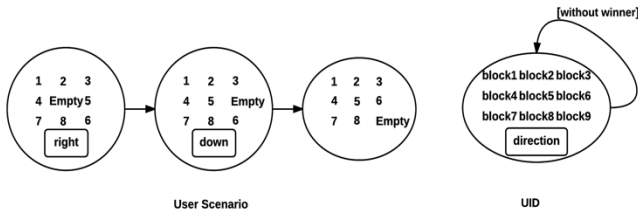


Figure 4. 8-puzzle US-UID (left) and UID (right)

Conceptually, following the classification of the FitNesse tabular format [11], the US-UID could be classified as test data and the UID could be classified as functional data. However, the symbols of US-UID (interaction states, user input, system output and transitions) must be synchronized with the respective symbols of a UID. For example, the user inputs "right" and "down", that appear in the US-UID shown in Figure 4, may be associated with the entry "direction" of the UID.

Definitions. In order to execute the US-UIDs defined by a user, the information represented in the US-UIDs has to be

synchronized with the functional data. To achieve that, the web framework considers the following definitions:

n = is the amount of interaction states belonging to US – UID

a = is the amount of user inputs belonging to $State_i$

b = is the amount of system outputs belonging to $State_i$

$value_i^y$ = is a user input belonging to $State_i$

$value_i^x$ = is a system output belonging to $State_i$

$model_i$ = is the functional data synchronized with $State_i$

$model_i^y$ = is the functional data synchronized with $input_i^y$

$model_i^x$ = is the functional data synchronized with $output_i^x$

$input_i^y = \{value_i^y, model_i^y\}$

$output_i^x = \{value_i^x, model_i^x\}$

$transition_i = \{source_i, target_i\}$

$target_i$ = is the interaction state of destination of a transition

$source_i$ = is the interaction state of origin of a transition

$Inputs_i = \{input_i^1, input_i^2, \dots, input_i^a\}$, where: $input_i^y \in Inputs_i \wedge 0 \leq y \leq a$

$Outputs_i = \{output_i^1, output_i^2, \dots, output_i^b\}$, where: $output_i^x \in Outputs_i \wedge 0 \leq x \leq b$

$State_i$ = is an interaction state

$\equiv \{Inputs_i, Outputs_i, transition_i, model_i\}$

US – UID = is a set of interactions states

$\equiv \{State_1, State_2, \dots, State_n\}$, where: $State_i$

$\in US - UID \wedge 0 < i$

$\leq n \wedge State_1$ is the start $\wedge State_n$ is the end

3. PROPOSAL AND IMPLEMENTATION

In the web framework for test automation we are proposing in this paper, the input format for the tests is US-UIDs. These scenarios represent the requirements of the application code. Thus, the US-UIDs should be run through the framework in order to assist the validation of requirements. Figure 5 shows the architecture of the proposed framework.

The framework architecture is based on FitNesse. The framework modules, shown as green in Figure 5, are: Scenario Editor, User Scenario Runner, and the SLIM¹ Executor. The extension points of the framework, shown as yellow in Figure 5, are: User Scenarios with Functional Data, Custom Fixtures, and the SUT.

¹<http://www.fitnessse.org/FitNesse.UserGuide.WritingAcceptanceTests.Slim>

The modules of the framework and extension points are presented in the following subsections, along with examples of the 8-puzzle game presented in Longo and Vilain [9].

3.1 Scenario Editor

The editor allows the creation of US-UIDs. A browser can be used to access the editor. The editor supports only the regressive method of creating US-UIDs. However, the main objective is the execution of US-UIDs as automated testing. Figure 6 shows the scenario editor.

The components of the editor are: control buttons (see Figure 6, top); the table to structure the scenarios (see Figure 6, center); and functional data (see Figure 6, right).

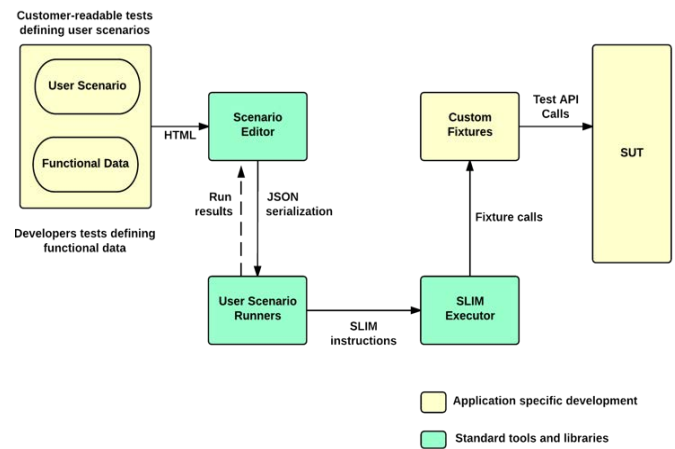


Figure 5. Framework architecture based on FitNesse

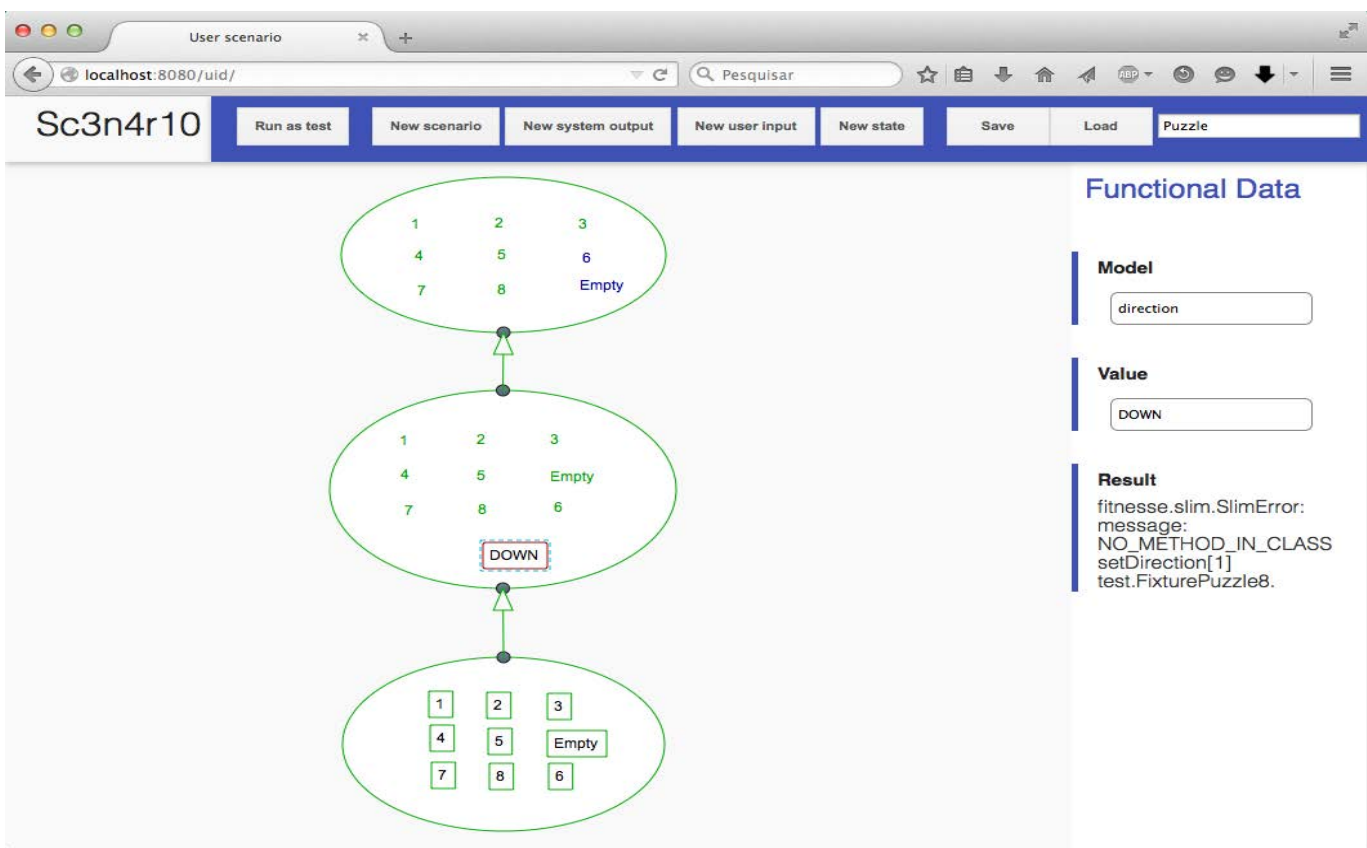


Figure 6. Scenario Editor

The control buttons have the following functions:

- Run as test - run the US-UID as a test;
- New scenario - create a new scenario;
- New system output - create a system output;
- New user input - create a user input;
- New state - create a new interaction state and a transition to the selected state.

The table is a simple board to present and edit the structures of US-UIDs. The implementation uses the library Draw2D² and manipulates the symbols of scenarios in accordance with the desired structure. The US-UIDs can be serialized to JSON³, allowing dispatch to the executor.

² Draw2D – <http://www.draw2d.org>

³ JSON – <http://json.org>

The functional data consist of the model, value and result. The model is the name of the fixture ($model_i$, $model_i^y$, $model_i^x$) and is equivalent to a given function in a FitNesse table (see Figure 1). The value is associated with the test data ($value_i^y$, $value_i^x$) of the selected symbol. The result is the state (success, error or failure) of the selected symbol execution.

3.2 User Scenario Runner

The User Scenario Runner is responsible for converting a US-UID in SLIM instructions. It receives as input a JSON file containing the US-UID and the functional data. Thus, the instructions are created according to the flow shown in the US-UID. The User Scenario Runner creates two types of SLIM instructions:

- `fitnesse.slim.instructions.MakeInstruction`: is used to create objects of a class. However, the class must match the fixture from an interaction state ($State_i$). This instruction is created for each interaction state of the US-UID;
- `fitnesse.slim.instructions.CallInstruction`: is used to call a method. This method corresponds to a system output ($output_i^x$), user input ($input_i^y$) or transition of states ($transition_i$).

After the execution of the instructions, the Scenario Editor presents the results. To facilitate the traceability, the results of instructions are classified as: success, error or failure. Table 2 presents the classification of the results of the instructions.

Table 2 - Classification of results for traceability.

Result	Color	Cause
Success	Green	The application code was executed correctly.
Error	Red	The application code was executed incorrectly.
Failure	Blue	The expected value is different from the result of the execution.

Failure is considered only when the result of a system output instruction ($output_i^x$) is different from the expected value ($value_i^x$).

3.3 SLIM Executor

The SLIM Executor is the responsible for invoking the instructions. To facilitate the implementation of accessory classes the following adjustments are made in the instructions:

- `MakeInstruction` ($State_i$): functional data ($model_i$) is replaced by the appropriate fixture class;
- `CallInstruction` ($input_i^y$): the prefix "set" is added to the name of the method ($model_i^y$);
- `CallInstruction` ($output_i^x$): the prefix "get" is added to the name of the method ($model_i^x$);
- `CallInstruction` ($transition_i$): the prefix "to" is added to the name of the method ($model_i^y$).

3.4 User Scenario and Functional Data

Conceptually, customers create the US-UIDs and developers define the functional data. So, in the example presented here the US-UIDs of the 8-Puzzle game are used as defined in [9]. Then, for each 8-Puzzle US-UID, the functional data are defined in the editor (see Figure 7, right). However, each US-UID requires an initial state for execution. This state may be different for each US-UID. In this case it is necessary to set an additional condition in the setup of the US-UID. Figure 7 shows an example of additional configuration.

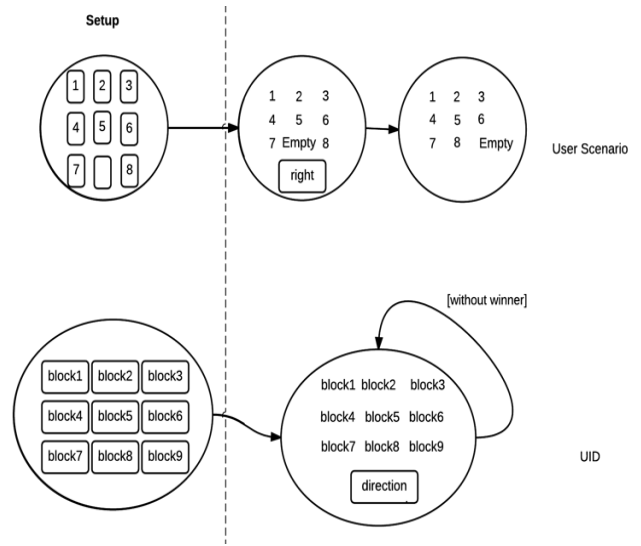


Figure 7. 8-Puzzle US-UID with additional configuration and UID

Frameworks of the family xUnit call the fixture setup of the configuration code to put the SUT in adequate conditions to exercise the tests [1, 16]. Similar to this concept, the additional interaction state (see Figure 7, left) contains the necessary preconditions for the execution of the US-UID.

3.5 Custom Fixtures

Fixtures are responsible for the communication between the US-UID and the SUT. The fixtures are constructed based on functional data. Figure 8 shows an example of a code for the fixtures of an 8-Puzzle US-UID (Figure 7).

```

1 package test;
2
3 @Fixture("Puzzle8")
4 public class FixturePuzzle8 {
5
6     private GamePuzzle8 sut = FixtureSetup.getGame();
7
8     // Transaction (without winner)
9     public void toPuzzle8() { }
10
11     // User input
12     public void setDirection(String direction) {
13         sut.moveEmpty(Directions.forName(direction));
14     }
15
16     // System output
17     public Object getBlock1() {
18         return sut.getBlock1();
19     }
20
21     public Object getBlock9() {
22         return sut.getBlock9();
23     }
24 }

```

Figure 8 - Example of a fixture class for exchange of information between US-UID and SUT

The SLIM Executor invokes the SUT through a fixture class. The fixture class is identified by the annotation `@Fixture` (see Figure 8, line 3). Each state interaction is associated with an accessory class and each user input is associated with a method "set" (see Figure 8, line 12) of the fixture class. Similarly, each output system is associated with a "get" method (see Figure 8, line 17) and each transition is associated with a method "to" (see Figure 8, line 9).

4. APPLICABILITY

But how to apply the proposed framework for testing web pages? To answer this question, we selected a web application as

example. The application is a simple page for authentication. For the construction of the application, we utilized the application server Tomcat⁴, JavaServer Faces⁵ (JSF), Selenium⁶, Java and the proposed framework. The construction steps of the testing are the following:

- Creation of US-UID;
- Appointment of fixtures of US-UID;
- Creation of classes of fixtures;
- Creation of web pages;
- Execution of the US-UID.

4.1 Creation of US-UID

The US-UID is built through the editor. Initially we built up the US-UID according to the application requirements. Figure 9 shows the US-UID to the authentication page.

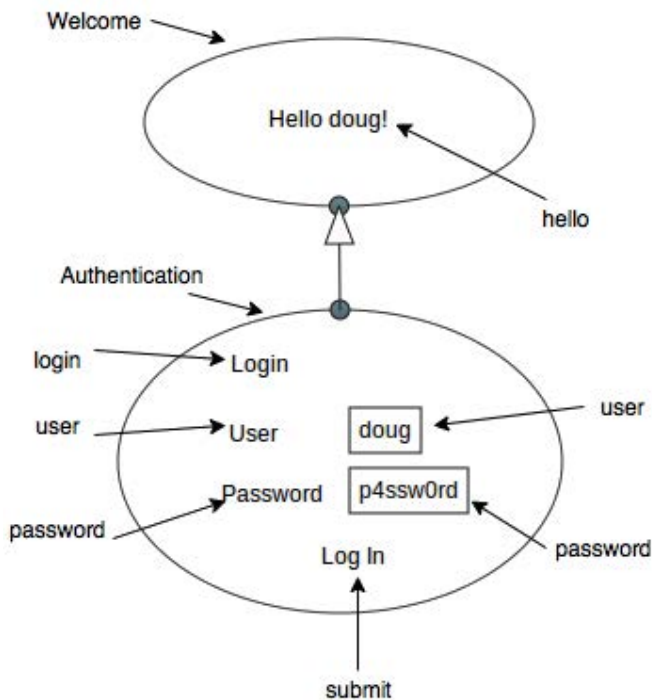


Figure 9 – Example of US-UID to an authentication page and appointment of fixtures.

The US-UID for the authentication page, shown in Figure 9, contains two interaction states:

- In the first interaction state (bottom, Figure 9) are shown the system outputs "Login", "User", "Password", "Log In" and the user inputs "doug", "p4ssw0rd";
- In the second state of interaction (top, Figure 9) only a system output is displayed: "Hello Doug!".

4.2 Appointment of fixtures of US-UID

The appointment of fixtures of the US-UIDs is made by the editor and these fixtures are displayed as functional data. Figure 9 shows

the US-UID with the appointment of fixtures. In this example, the names of the fixtures are displayed outside the interaction states (ellipses) and are connected with the elements of the US-UID by an arrow, for example, the fixture "Welcome" is connected to the second interaction state (ellipse).

4.3 Creation of classes of fixtures

The fixtures are responsible for the exchange of information between the US-UID and the browser. For the US-UID shown in Figure 9, it is required two classes: *FixtureAuthentication* and *FixtureWelcome*. Figure 10 shows the class *FixtureAuthentication*.

```

1 package test.br.authentication;
2
3 import org.openqa.selenium.By;
4
5
6
7
8
9 @Fixture("Authentication")
10 public class FixtureAuthentication {
11     private static WebDriver drive;
12
13     public FixtureAuthentication() {
14         drive = new FirefoxDriver();
15         String address = "http://localhost/auth";
16         drive.navigate().to(address);
17     }
18
19     public void toWelcome() {
20         By id = By.id("authentication:submit");
21         drive.findElement(id).click();
22     }
23
24     public void setPassword(String password) {
25         By id = By.id("authentication:password");
26         drive.findElement(id).sendKeys(password);
27     }
28
29     public String getUser() {
30         By id = By.id("authentication:userTitle");
31         return drive.findElement(id).getText();
32     }
33
34     public void setUser(String user) {}
35
36
37
38     public String getPassword() {}
39
40
41     public String getSubmit() {}
42
43
44     public String getLogin() {}
45
46
47
48
49
50
51
52
53
54     public static WebDriver getWebDriver() {}
55 }

```

Figure 10 - Class FixtureAuthentication

The class *FixtureAuthentication* is annotated with *@Fixture* (line 9). The annotation makes the connection between the class and the interaction state. In the class constructor is created an object *drive* (line 14), which is the Web browser automation. Once created, the drive allows access to the web application address (line 16). The method *toWelcome* (line 19) is responsible for exchanging the interaction state and sends click to the button. Figure 13 displays the authentication page and identifiers for each element. The method *setPassword* (line 24) receives a user input and sends this information to the page. The method *getUser* (line 29) obtains the page information and returns it to the US-UID. The following methods (lines 34, 39, 44, 49) are responsible, in a

⁴ <http://tomcat.apache.org/>

⁵ <http://www.oracle.com/technetwork/java/javase/javaserverfaces-139869.html>

⁶ <http://www.seleniumhq.org/>

similar way, for exchanging information. The method *getWebDrive* (line 54) returns the object *WebDrive* to share with other objects. The class *FixtureWelcome* is also responsible for the exchange of the information between the US-UID and the browser. Figure 11 shows the code of the class *FixtureWelcome*.

```
1 package test.br.authentication;
2
3 import org.openqa.selenium.By;
4
5
6 @Fixture("Welcome")
7 public class FixtureWelcome {
8     private WebDriver drive;
9
10    public FixtureWelcome() {
11        drive = FixtureAuthentication.getWebDriver();
12    }
13
14    public String getHello() {
15        By id = By.id("hello");
16        return drive.findElement(id).getText();
17    }
18 }
19
20 }
```

Figure 11 - Code of the class *FixtureWelcome*

The class *FixtureWelcome* receives the annotation *@Fixture* (line 8) with the value "Welcome" that enables the communication with the latest interaction state of US-UID. In the class constructor (line 13) is obtained the *WebDrive* used in class *FixtureAuthentication*. The method *getHello* (line 16) is responsible for retrieving the page text of the element *hello*. To retrieve the text is created an element (line 17) and the text is returned through the drive.

4.4 Creation of web pages

For the US-UID two pages were created using the framework JSF and the application server Tomcat. The authentication page is related to the first interaction state (bottom, Figure 9) and the welcome page is related to the second interaction state (top, Figure 9). Figure 12 shows the authentication page code.

```
1 <html xmlns="http://www.w3.org/1999/xhtml"
2     xmlns:f="http://java.sun.com/jsf/core"
3     xmlns:h="http://java.sun.com/jsf/html">
4 <head>
5 </head>
6 <body>
7 <h:form id="authentication">
8 <h1 id="loginTitle">Login</h1>
9 <h:panelGrid columns="2">
10 <h:outputLabel id="userTitle" value="User" />
11 <h:inputText id="user" value="#{user}" />
12 <h:outputLabel id="passwordTitle" value="Password" />
13 <h:inputSecret id="password" value="#{password}" />
14 <br />
15 <h:commandButton id="submit"
16     action="#{beanAuthentication.login(user,password)}"
17     value="Log In" />
18 </h:panelGrid>
19 </h:form>
20 </body>
21 </html>
```

Figure 12 – XHTML code of authentication page

The authentication page code contains a form (line 14), the title "Login" (line 15), a grid of two columns (line 16) with user authentication attributes and a button to submit the form. The submission button of the form invokes the login method of the object *beanAuthentication*. This object is written in a Java class and is responsible for controlling the authentication. Figure 13 shows the authentication page produced by the browser and the *ids* of the page elements.

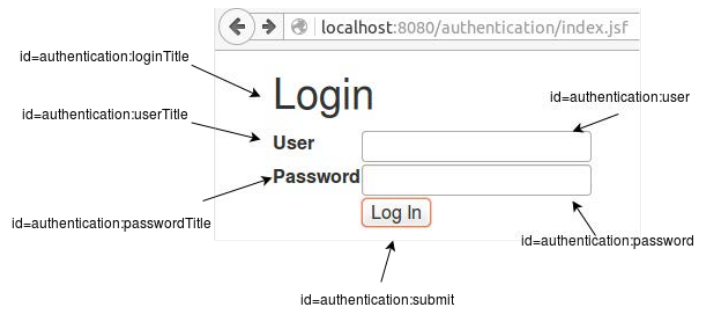


Figure 13 - Authentication page and the identifiers of the elements

The identifiers of the elements are displayed next to the elements in the page and are indicated by arrows. The fixtures locate the page elements through identifiers. After locating the elements, it is possible to exchange information with the browser. The automated communication by the fixture is similar to the conventional use of the browser, where a person fills in information and clicks the buttons.

Figure 14 shows the code of the welcome page. This page contains only a text with a welcome message to the authenticated user. The message is displayed by formatted text (line 14).

```
1 <html xmlns="http://www.w3.org/1999/xhtml"
2     xmlns:f="http://java.sun.com/jsf/core"
3     xmlns:h="http://java.sun.com/jsf/html">
4 <head>
5 </head>
6 <body>
7 <h:outputFormat value="Hello {0}!" id="hello">
8     <f:param value="#{beanAuthentication.user}" />
9 </h:outputFormat>
10 </body>
11 </html>
```

Figure 14 – Code XHTML of welcome page

4.5 Execution of the US-UID

The execution of the US-UID is performed by the proposed framework, however, the web application testing requires the web browser automation (Selenium). Figure 15 shows the dependency of the artifacts.

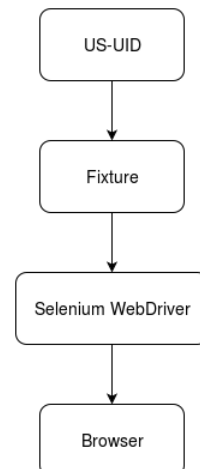


Figure 15 - Diagram of dependence of artifacts for web application testing.

5. EVALUATION

The evaluation of the proposed web framework is the result of its traceability capacity: success, error and failures in automated testing. For evaluation and comparison, we used a module of a Web-based application developed with 232 test cases that are automated with JUnit⁷.

The Web-based application was developed to evaluate and follow up the e-Tec Brazil network courses [16]. The application was developed in Java for Web technologies, and with the DBMS PostgreSQL. The development team and clients used the US-UIDs for communication and collaboration while developing some modules of the Web-based application. The US-UIDs were drawn in paper and then manually deployed as test cases for JUnit. Some test cases used Selenium to exchange information with the browser. The module of application used in the experiment contains 4 US-UIDs. Table 3 shows the characteristics of the 4 US-UIDs. The manual implementation of the scenarios resulted in 232 test cases that were automated with JUnit.

Table 3 - Characteristics of the US-UIDs used in the experiment.

US-UID	#Interaction state	#User input	#System output	#Transition
A	3	4	25	2
B	5	5	37	4
C	7	6	48	6
D	8	7	57	7

After, the same US-UIDs were created using the proposed framework. The proposed framework resulted in 231 test cases generated from the 4 US-UIDs (see Table 3) used in the experiment. The proposal considers that each user input, system output, interaction of state and transition corresponds to a test case.

The evaluation of the experiment consists in comparing the result of the execution of the 232 test cases manually implemented and the 231 test cases generated from the proposed framework. This experiment did not investigate whether there is a relationship between the amounts of test cases.

Initially, the experiment consisted in the execution of the test cases. In both approaches, all test cases resulted in success. After that, changes were made in the application code using different techniques to generate two experiments: code mutation [17,18, 19] and lack of code [20].

The experiment using code mutation was conducted by manually changing the value of an array in the application code. Figure 16 shows the result of the experiment using code mutation.

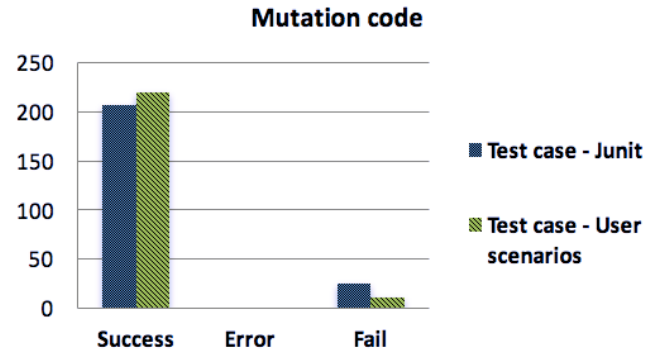


Figure 16 - The results of traceability for: success, errors, and failures for mutation code.

As observed in Figure 16, the results are divided into test cases of JUnit that were manually deployed and test cases of US-UIDs that were automatically generated by the proposed framework. The experiment using code mutation with test cases of JUnit resulted in 207 successes and 25 failures, while the experiment with the test cases of US-UIDs resulted in 220 successes and 11 failures. Both did not track errors in the code.

The experiment using lack of code was performed by removing a class from the application code. Figure 17 shows the result of the experiment using lack of code.

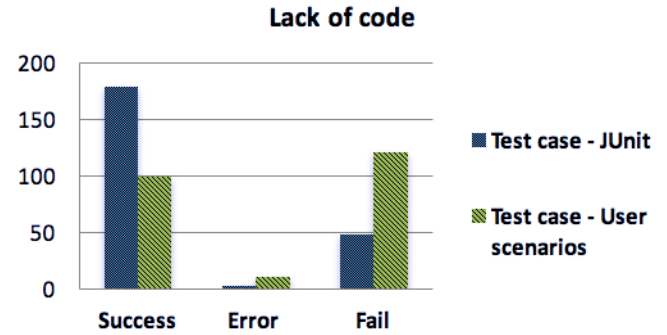


Figure 17 - The results of traceability for: success, errors, and failures for lack of code.

As observed in Figure 17, the results are also divided into test cases of JUnit that were manually deployed and test cases of US-UIDs that were automatically generated by the proposed framework. The experiment using lack of code with tests cases of JUnit resulted in 179 successes, 48 faults and 3 errors, while the experiment with test cases of US-UIDs resulted in 99 successes, 121 failures and 11 errors. However, the framework JUnit did not identify 2 of the 232 test cases manually deployed. This occurred because of a dependency between the two test cases and the class that was removed.

6. VALIDITY

Internal validity refers to the causal inferences made based on experimental data [21]. Thus, to determine the validity of our proposal we used the 8-puzzle game and a web system. The 8-puzzle game is a toy example used to explain the proposed framework, while the web system is an information system used in the experiments. In the experiments, the goal was only to determine whether the framework could determine success, error or failure in the application code through the execution of the US-UIDs.

⁷ <http://junit.org>

External validity refers to the ability to generalize the findings to other domains [21]. With our empirical knowledge, we recommend to use the web framework to build scenarios for domains as information systems. However, in areas such as first-person games, it can be difficult to represent the information exchange between the user and the system by US-UIDs.

7. DISCUSSION

The paper presents a framework for test automation based on the use of US-UIDs for user scenarios descriptions. From US-UIDs, test cases are extracted to be executed on the framework FitNesse. The goal is to support Automated Acceptance Testing in the Agile Software Development Context. However, three discussions should be made:

- Format of the US-UID;
- Evaluation results;
- Use of the framework.

The format of the US-UID is defined in [9] [15]. The format of the US-UID is different from the table format of the FitNesse or other acceptance test formats like BBD. However, the paper explores US-UID for test automation, leaving outside many considerations about the format.

The evaluation of the work compared the proposed framework and JUnit. In the evaluation we performed a number of tests effectively to compare the two frameworks. However, both frameworks use distinct formats for the tests. Thus, the test cases were normalized, allowing make comparisons of the results of execution.

The use of the proposed framework consists of two parts: the creation of scenarios and execution of scenarios. To create the scenarios the editor that is accessed by the browser was used. Thus, the editor facilitates the sharing of scenarios, such as: two people from different computers may be creating scenarios. However, the execution of the scenario in the browser is a little complicated because the browser shows the result of the error and the developer has to search the error in code. Thus, an improved alternative is to integrate the framework proposed to the code editor, facilitating the traceability of error codes.

8. CONCLUSION

The ATDD approach considers the development of tests a priori to the application code. In the context of ATDD, this paper presented a web framework for test automation from US-UID. The proposed framework was explained through the 8-puzzle game [9].

Two experiments using a Web-based application were carried out to evaluate the traceability capacity of test cases using the framework proposed in this work and the framework JUnit. We observed that US-UID can be converted to automated test cases that preserve the properties of traceability, then allowing the framework evaluates the success, error or failure of the application code. These experiments have shown that small changes in the application code are captured by both frameworks. Thus, US-UIDs can be used to determine whether the application code is appropriate and fulfills the customer requirements.

The proposed framework executes the US-UIDs without the need for developers to convert them manually to another test framework. With the proposed framework the technique US-UID becomes an important option to include the end user as active participant of the requirements specification using the ATDD approach.

The main contributions of this study were the implementation and evaluation of a web framework for automation of US-UIDs. Future research will investigate the time and effort of test automation using US-UIDs; and metrics for evaluating the evolution of US-UIDs.

We would like to thank the e-Tec Brazil development team for collaboration on data for evaluation of the proposed framework.

9. REFERENCES

- [1] Beck, K. 2003. *Test Driven Development: By Example*. Addison-Wesley.
- [2] Andersson, J., Geoff, B., and Peter S. 2003. XP with Acceptance-Test Driven Development: A rewrite project for a resource optimization system. In *Extreme Programming and Agile Processes in Software Engineering*. Springer Berlin Heidelberg, 180-188. DOI= http://dx.doi.org/10.1007/3-540-44870-5_23
- [3] Hayes, J. H., Alex, D., and David, S. J. 2009. Towards traceable test-driven development. In *Traceability in Emerging Forms of Software Engineering, 2009. TEFSE'09. ICSE Workshop on. IEEE*, 26-30. DOI= <http://dx.doi.org/10.1109/TEFSE.2009.5069579>
- [4] Haugset, B., and Hanssen, G. K. 2008. Automated acceptance testing: A literature review and an industrial case study. In *Agile, 2008. AGILE'08. Conference. IEEE*, pp.27-38. DOI= <http://dx.doi.org/10.1109/Agile.2008.82>
- [5] Miller, R., and Collins, C. T. 2001. Acceptance testing. In *Proc. XPUniverse 2001*.
- [6] Dybå, T., and Torgeir, D. 2008. Empirical studies of agile software development: A systematic review. In *Information and software technology*, vol. 50, n.9. Elsevier. 833-859. DOI= <http://dx.doi.org/10.1016/j.infsof.2008.01.006>
- [7] Hanssen, G. K., and Haugset, B. 2009. Automated acceptance testing using fit. In *System Sciences, 2009. HICSS'09. 42nd Hawaii International Conference on System Sciences. IEEE*. 1-8. DOI= <http://dx.doi.org/10.1109/HICSS.2009.83>
- [8] Alvestad, K. 2007. *Domain Specific Languages for Executable Specifications*. Institutt for datateknikk og informasjonsvitenskap.
- [9] Longo, D. H., and Vilain, P. 2015. Creating User Scenarios through User Interaction Diagrams by Non-Technical Customers. In *27th International Conference on Software Engineering and Knowledge Engineering*. 330-335. DOI= <http://dx.doi.org/10.18293/SEKE2015-179>
- [10] Borg, R., and Kropp, M. 2011. Automated acceptance test refactoring. In *Proceedings of the 4th Workshop on Refactoring Tools*, ACM. 15-21. DOI= <http://dx.doi.org/10.1145/1984732.1984736>
- [11] Druk, M. and Kropp, M. 2013. ReFit: A Fit test maintenance plug-in for the Eclipse refactoring plug-in. In *Developing Tools as Plug-ins (TOPI), 3rd International Workshop on developing tools as plug-ins. IEEE* 2013. pp. 7-12. DOI= <http://dx.doi.org/10.1109/TOPI.2013.6597187>
- [12] Güell, N., Schwabe, D., and Vilain, P. 2000. Modeling interactions and navigation in web applications. In *Conceptual Modeling for E-Business and the Web*. Springer. 115-127. http://dx.doi.org/10.1007/3-540-45394-6_11

- [13] Zeferino, N. V., and Vilain, P. 2014. A model-driven approach for generating interfaces from user interaction diagrams. In *Proceedings of the 16th International Conference on Information Integration and Web-based Applications & Services*. ACM, 474-478. DOI= <http://dx.doi.org/10.1145/2684200.2684326>
- [14] Greiler, M., Deursen, A., and Storey, M. 2013. Automated detection of test fixture strategies and smells. In *Software Testing, Verification and Validation (ICST), IEEE Sixth International Conference on*. IEEE. 322-331. DOI= <http://dx.doi.org/10.1109/ICST.2013.45>
- [15] Longo, D. H., and Vilain, P. 2015. User scenarios through user interaction diagrams. In *International Journal of Software Engineering and Knowledge Engineering*. v. 25, n. 09n10, 1771-1775. DOI= <http://dx.doi.org/10.1142/S0218194015710151>
- [16] Cislighi, R., Wilges, B., Nassar, S. M., Longo, D. H., and Mateus, G. P. 2014. Avaliação de polos sob uma perspectiva georreferenciada. In *XI Congresso Brasileiro de Ensino Superior a Distância (ESUD'14)*. Florianópolis. 771-781.
- [17] Papadakis, M., Malevris, N., and Kallia M. 2010. Towards automating the generation of mutation tests. In *Proceedings of the 5th Workshop on Automation of Software Test*. ACM. 111-118. DOI= <http://dx.doi.org/10.1145/1808266.1808283>
- [18] Arcuri, A., Iqbal, M. Z., and Briand, L. 2010. Black-box system testing of real-time embedded systems using random and search-based testing. In *Testing Software and Systems*. Springer. 95-110. DOI= http://dx.doi.org/10.1007/978-3-642-16573-3_8
- [19] Meinke, K., and Niu, F. 2010. A Learning-Based Approach to Unit Testing of Numerical Software. In *IFIP International Conference on Testing Software and Systems*. Springer Berlin Heidelberg. 221-235. DOI= http://dx.doi.org/10.1007/978-3-642-16573-3_16
- [20] Thongtanunam, P., Tantithamthavorn C., Kula, R. G., Yoshida, N. Iida, H., and Matsumoto, K. 2015. Who should review my code? A file location-based code-reviewer recommendation approach for Modern Code Review. In *Software Analysis, Evolution and Reengineering (SANER), 2015 IEEE 22nd International Conference on*. IEEE. 141-150. DOI= <http://dx.doi.org/10.1109/SANER.2015.7081824>
- [21] Yin, R. K. 2003. *Case Study Research: Design and Methods*. 3rd ed., ser. Applied Social Research Methods Series, L. Bickman and D. J. Rog, Eds. Sage Publications, vol. 5.